DATABASE SYSTEMS

Relational Algebra Lab

Student Tutorial and Practical Guide

What you need A browser, the supplied dataset, the submission template, and a plain-text editor.

Lab length: ~ 30 minutes | Main work: 5 practical questions

Use this guide in order. The worked examples are for practice. They do not give away the answers to the required questions.

1. Quick start

Today you will write and run relational algebra expressions in RelaX. The main idea is simple: each operator takes one or more relations and produces another relation.

RA in one sentence: SQL usually states the result you want. Relational algebra shows the operations used to build that result.

The five-step routine

Read the output → Choose relations → Build inside out → Run and inspect → Submit clearly

Before you begin

[ ] I have lab_dataset.txt.

[ ] I have opened RA_Lab_Submission_Template.txt in a plain-text editor.

[ ] I know where I will save my final work.

[ ] I can open the lab assignment in Canvas.

A few rules that save time

- Read nested expressions from the inside outward.
- Use explicit join conditions unless you have checked every common attribute name.
- Before union, intersection, or difference, make sure both inputs have the same columns and compatible types.
- Relations are unordered. A different row order is not a different answer.
- Do not submit screenshots only. Submit the expression as text.

2. Open RelaX and load the dataset

Open: https://dbis-uibk.github.io/relax/

1. Choose Create new Dataset or open the Group Editor. The wording may be slightly different depending on the screen size.
2. Open the supplied dataset text file and copy the whole file.
3. Paste the text into the dataset editor.
4. Save or load the group named Campus Collaboration Lab.
5. Open the Relational Algebra editor.
6. Type Student and press Ctrl+Enter (or Ctrl+Return).
7. Check that the Student relation appears with 13 tuples.

Setup check: If Student runs, the dataset is ready. Do not continue if the editor says that Student is unknown. Go back and make sure the complete dataset text was pasted and loaded.

Expected relation sizes

Relation	Tuples	Relation	Tuples
Student	13	Course	8
Enrollment	30	Club	5
Membership	20	Project	8
WorksOn	20		

If the website does not run

- Refresh the page once and reload the dataset.
- Try a private/incognito window or a different browser.
- Check that quotes are straight quotes, for example 'C101', not curved quotation marks.
- Use the supplied dataset text file instead of retyping the tables.

3. Know the dataset

You do not need to memorize every row. You do need to know which relation contains each attribute and how the relations connect.

Relation	Attributes	What it stores
Student	sid, sname, dept, year	One row per student
Course	cid, title, dept, level	One row per course
Enrollment	sid, cid, term, grade	Students taking courses
Club	club_id, club_name, category	One row per club
Membership	sid, club_id, role	Students joining clubs
Project	pid, ptitle, theme, lead_sid	Projects and their leads
WorksOn	sid, pid, hours	Students working on projects

Key connections

```
Student.sid  --> Enrollment.sid
Course.cid   --> Enrollment.cid
Student.sid  --> Membership.sid
Club.club_id --> Membership.club_id
Student.sid  --> WorksOn.sid
Project.pid  --> WorksOn.pid
Student.sid  --> Project.lead_sid
```

Important data details: Two students are named Alex Lee. Some students have no club. One student has no enrollment. Student and Course both contain an attribute named dept. These details are intentional and matter in several questions.

4. RelaX syntax you will use

You may type the plain-text form. RelaX can also display the mathematical symbols.

Idea	Math	RelaX text	Use it for
Selection	σ	sigma	Keep rows that meet a condition
Projection	π	pi	Keep selected columns
Rename	ρ	rho	Give a relation or attribute a new name
Union	∪	union	Rows in either compatible relation
Intersection	∩	intersect	Rows in both compatible relations
Difference	−	-	Rows in the first relation but not the second
Cross product	×	cross join	Pair every row from two relations
Join	⋈	join	Combine related rows

Small syntax examples

Selection: keep DATA students

```
sigma dept = 'DATA' (Student)
```

Projection: keep only student ID and name

```
pi sid, sname (Student)
```

Rename and join: give each input a short alias

```
rho S (Student) join S.sid = E.sid rho E (Enrollment)
```

Set difference: course IDs that never appear in Enrollment

```
pi cid (Course) - pi cid (Enrollment)
```

Read from the inside out

```
pi sid, sname (
  sigma dept = 'DATA' and year >= 3 (Student)
)
```

First select DATA students in year 3 or 4. Then keep only sid and sname.

Selection and projection are easy to mix up: RA selection (sigma) keeps rows. RA projection (pi) keeps columns. In SQL, WHERE behaves like RA selection, while the SELECT list behaves like RA projection.

5. Worked examples

Run these examples before starting the marked questions. They show the usual pattern: filter, join if needed, then project the final columns.

Example A: selection followed by projection

Task: return the IDs and names of students in the DATA department.

Mathematical RA: π_sid,sname(σ_dept='DATA'(Student))

RelaX:

```
pi sid, sname (
  sigma dept = 'DATA' (Student)
)
```

Check: the result should contain S03 Cara Singh, S05 Erin Park, and S10 Jae Patel.

Example B: join, selection, and projection

Task: show each student who works more than 8 hours on a project. Return sid, sname, pid, and hours.

```
pi S.sid, S.sname, W.pid, W.hours (
  sigma W.hours > 8 (
    rho S (Student) join S.sid = W.sid rho W (WorksOn)
  )
)
```

Check: the output has seven tuples. Click the join and selection nodes in the operator tree to see how the relation becomes smaller.

Example C: a “not exists” pattern

Task: return course IDs that have no enrollment.

```
pi cid (Course)
-
pi cid (Enrollment)
```

Check: the result is C108. The first relation is the full set of course IDs. The second is the set of course IDs that appear in Enrollment. Subtracting the second from the first leaves courses with no matching enrollment.

Patterns to remember

Words in the question	Useful RA pattern
and	One selection predicate with and, or an intersection when comparing two derived sets
or (or both)	Union of two compatible result relations
both	Intersection or a self-join
not / no related row	Start with the valid universe, then subtract the matched set
at least two different	Use two renamed copies and require different item IDs
every / all	Eligible set minus rows with a counterexample

Natural join warning: Natural join matches every attribute name shared by the two inputs. It does not know the intended foreign keys. In this dataset, Student and Course both have dept, so an unplanned natural join can remove valid enrollments.

6. Lab Exercises

Spend 4 minutes on setup, 24 minutes on the five queries, and 2 minutes checking and submitting your file. Use the campus showcase scenario for each query.

Q1 Club invitations

Find students in AI Society (K1), Robotics Club (K2), or both. Use union and return each student once.

Return sid, sname

Q2 Two students named Alex Lee

Find the courses taken by either Alex Lee where the course department differs from that student's department. Keep the two students distinguishable.

Return sid, sname, cid, title

Q3 Projects to check in on

Find projects with no single work log of 10 hours or more. Include projects with zero work logs.

Return pid, ptitle

Q4 Showcase mentors

Find students who logged at least 5 hours on each of at least two different projects. Apply the threshold to each work log.

Return sid, sname

Q5 Academic recognition

Find students who have at least one enrollment and earned at least 80 in every enrollment.

Return sid, sname

7. Save and submit your answers

Create one plain text file

1. Download RA_Lab_Submission_Template.txt from the assignment. Open it in Notepad, VS Code, or another plain-text editor. Save a copy as RA_Lab_YOURSTUDENTID.txt in UTF-8 encoding. Replace YOURSTUDENTID with your own ID in both the filename and the Student ID line.

2. Keep [Q1] through [Q5] and all section labels. Under [RA], write the mathematical expression. Under [RELAX], paste the exact query you ran in RelaX. Either expression may span multiple lines.

3. Under [RESULT], keep the supplied comma-separated header. Copy the final result, with one row per line and commas between values. Include every result row and keep the column values in the same order as the header.

Example using a practice query

```
[RESULT]
sid,sname
S03,Cara Singh
S05,Erin Park
S10,Jae Patel
```

This example shows the format for the DATA department practice query. Enter your own result rows in the five question blocks.

Upload to Canvas

4. In Canvas, open your course, select Assignments, and open this lab. Choose Start Assignment and select File Upload if prompted. Choose your completed .txt file, then select Submit Assignment.

5. Check the submission confirmation and confirm the filename. If Canvas allows a new attempt, use it to replace an incorrect upload.

Final check: One UTF-8 .txt file, your student ID, five completed blocks, and the Canvas confirmation.

8. Troubleshooting and reference

If a query fails, run each inner expression separately. Check the join keys and aliases, then compare the columns on both sides of each set operation. Use the operator tree to find the first intermediate result that differs from what you expect.

Common RelaX patterns

Need	Pattern
Filter rows	sigma condition (R)
Choose columns	pi a, b (R)
Rename a relation	rho X (R)
Join on keys	rho A (R) join A.key = B.key rho B (S)
Either set	A union B
Both sets	A intersect B
In A but not B	A - B
All pairs	A cross join B

Checks before running

[ ] Are all attribute names available at the point where I use them?

[ ] Did I join on the correct key attributes?

[ ] Did I use aliases when a relation appears twice?

[ ] Are set-operation inputs compatible?

[ ] Did I project the requested output columns only?

[ ] Did I avoid an unintended natural join?

Useful translation

SQL idea	RA idea
WHERE condition	Selection (sigma)
SELECT column list	Projection (pi)
JOIN ... ON ...	Theta/equi-join
UNION	Union
INTERSECT	Intersection
EXCEPT	Set difference
DISTINCT	Built into pure RA set semantics

Work inside out. Check the schema at every step. Keep your final answer simple.

Based on Lecture 03: Relational Algebra and the Campus Collaboration lab materials.